Lecture Title: Spectral Clustering, Graph Clustering, and Semi-Supervised Clustering

9: Advanced Cluster Analysis

Duration: 1 Hour

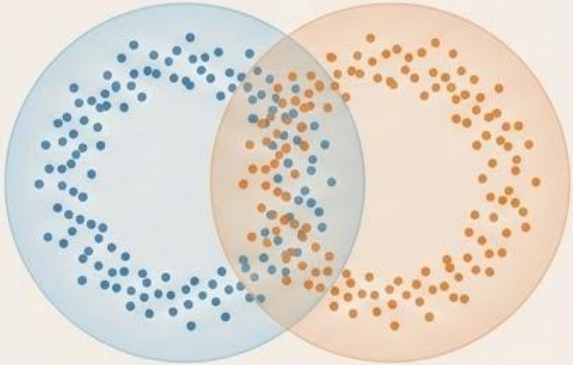
Target Audience: Data Science/Computer Science students

Prerequisites: Understanding of basic clustering, linear algebra (eigenvalues/vectors), graph theory fundamentals

Introduction to Advanced Clustering

When Traditional Methods Fail

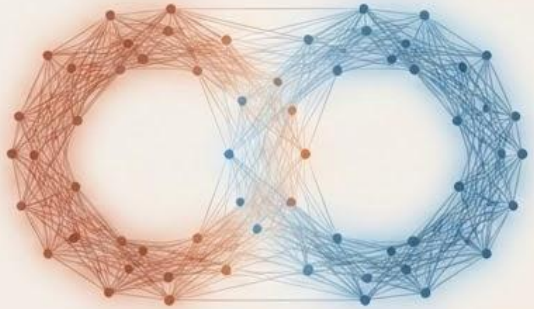
The Complex Geometry Challenge



The Scenario: Imagine trying to cluster data that is shaped like two intertwined rings.

The Limitation: Traditional partitioning algorithms like *k*-means fail completely here because they assume clusters are neat, convex spheres.

The Paradigm Shift & Solution



The Paradigm Shift: What if we stop looking at spatial coordinates and instead think of our data points as a graph, where the edges represent the similarity between points?

The Solution: We can use advanced methods—like spectral clustering—that rely on the eigenvectors of a similarity matrix to uncover these highly complex, non-linear shapes.

Learning Objectives

Spectral, Graphs, and Supervision

- Understand Nonnegative Matrix Factorization (NMF) and how it applies to clustering.
- Master the core concepts and algorithmic steps of spectral clustering.
- Learn specialized clustering methods designed strictly for graph and network data.
- Explore semi-supervised clustering techniques that leverage partial labels and user constraints.
- Understand how these methods apply to real-world domains like social networks, bioinformatics, and recommendation systems.

Introduction to NMF

The Concept and Mathematics

The Core Concept

NMF is a dimensionality reduction technique that factorizes a large, nonnegative data matrix into two smaller, nonnegative matrices.

The Mathematical Formulation

$$V_{n \times m} \approx W_{n \times k} \times H_{k \times m}$$

The Components

- V: The original data matrix (n objects $\times$ m features).
- W: The basis matrix (n objects $\times$ k latent components).
- H: The coefficient matrix (k latent components $\times$ m features).

The Strict Constraint

- All entries in V, W, and H must be nonnegative (greater than or equal to zero).

Why Nonnegativity Matters

Building with Additive Parts

Why impose this strict “no negative numbers” rule?

- **Parts-Based Representation**

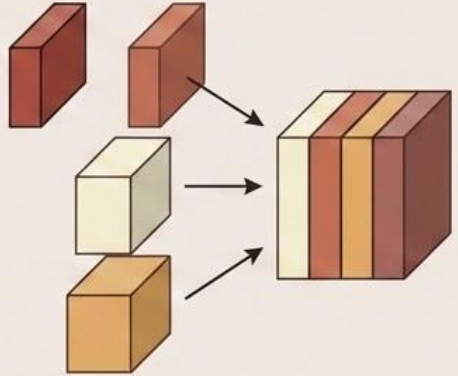
  Unlike standard methods that allow complex additions and subtractions of data, NMF forces the system to learn purely additive components.

- **Natural Interpretability**

  Because there are no negative weights to cancel things out, the resulting components are highly interpretable to humans. You are only enly ever adding parts together to build the whole.

- **Broad Applicability**

  This additive nature perfectly mirrors real-world data like images (adding pixels), text (adding word counts), and biology (adding gene expressions).



NMF for Clustering

Decoding the Factorized Matrices

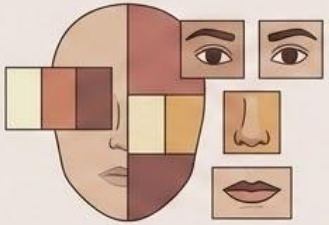
When we apply NMF specifically for clustering, the parameter k represents our desired number of clusters. The resulting matrices give us our assignments:

- **The Coefficient Matrix (H):**
  Each column in H acts as a soft indicator for cluster membership. membership. It tells us how strongly a specific feature or data point aligns with each of the k clusters.
- **The Basis Matrix (W):**
  Each row in W represents the cluster centers situated “within this new, reduced latent space.

Real-World Applications of NMF

Where Additive Clustering Excels

Image Analysis

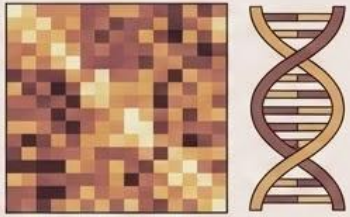


When applied to faces, NMF naturally isolates individual parts (eyes, noses, mouths) as distinct latent components.

Text Mining

Used for topic discovery by factorizing document-term matrices (grouping documents by adding together distinct, underlying topics).

Bioinformatics



Excellent for discovering complex, overlapping gene expression patterns without negative artifacts.

NMF thrives in domains where data is naturally composed of non-overlapping parts:

Optimization Techniques

How the Algorithm Learns

Finding the perfect W and H is computationally challenging. We rely on iterative optimization algorithms to approximate the solution:

- **Multiplicative Update Rules:** The classic approach (introduced by Lee & Seung, 1999) that iteratively updates the matrices while guaranteeing they remain nonnegative.
- **Alternating Nonnegative Least Squares (ANLS):** A faster, more modern approach that solves for W while holding H fixed, and then solves for H while holding W fixed.

Convergence Warning: Like k-means, these optimization methods guarantee convergence, but only to a local optimum. Multiple restarts are often required for the best results.

Spectral Clustering

A Graph-Based Approach to Complex Data

The Core Idea:

We use the eigenvectors of a similarity (or affinity) matrix to perform dimensionality reduction before clustering the data in a new, simpler space.

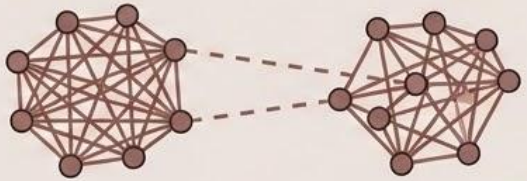
The Problem with Traditional Methods:

Standard algorithms like k-means fail completely when clusters are non-convex, arbitrarily shaped, or intertwined.

The Spectral Intuition:

Stop thinking of data as isolated points in space.

Instead, construct a graph where nodes are data points and edges represent the similarity between them.



The goal is to find clusters that minimize the “graph cut” (meaning we want very few edges connecting different clusters, and many edges within the same cluster).

The Spectral Algorithm: Part 1

Building the Mathematical Foundation

Step 1: Construct the Similarity Graph (W)

- **ϵ-neighborhood graph:** Connect points if they are within distance ϵ.
- **k-nearest neighbor graph:** Connect each point to its k closest neighbors.
- **Gaussian Kernel (Fully Connected):** Connect everything, weighting by distance:

$$w_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$$

Step 2: Compute the Graph Laplacian

- Calculate the **Degree matrix (D):** A diagonal matrix where $D_{ii} = \sum_j w_{ij}$
- Calculate the **Unnormalized Laplacian (L):** $L = D - W$
- Calculate the **Normalized Laplacian** (L_{sym}) (generally preferred):

$$L_{sym} = D^{-1/2} L D^{-1/2}$$

The Spectral Algorithm: Part 2

The Spectral Space and Final Assignment

Step 3: Compute Eigenvectors

Find the smallest k eigenvectors of the Laplacian matrix L (ignoring the trivial zero eigenvalue).

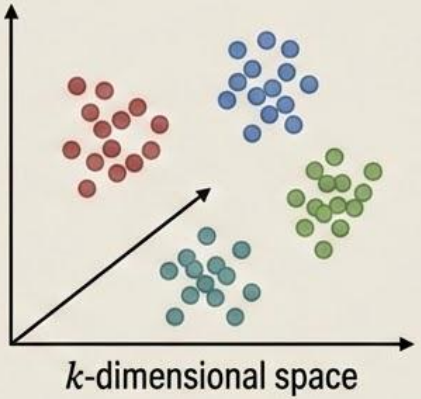
$$\left[v_1 \right], \left[v_2 \right], \cdots, \left[v_k \right]$$

Stack these eigenvectors side-by-side to form a new matrix: $U \in \mathbb{R}^{n \times k}$

$$U = \left(\left[v_1 \right], \left[v_2 \right], \cdots, \left[v_k \right] \right)$$

Step 4: Cluster in Spectral Space

Each row of matrix U now represents a data point embedded in a new k-dimensional space.



Apply standard k-means clustering to these rows.

The original data points inherit the cluster labels found in this new space.

Why Spectral Clustering Works

Unwrapping Complex Geometry

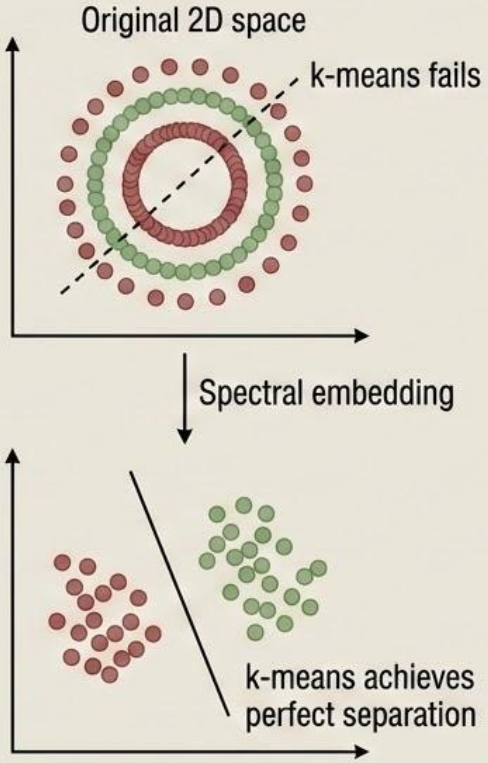
The Mechanism

The eigenvectors reveal hidden cluster structures by embedding points that are strongly connected in the graph closely together in the new space.

The Classic Example: Two Intertwined Rings

- **Original 2D space**: Two concentric circles or intertwined rings. k-means fails.
- **Spectral embedding**: The graph connectivity forces the points from the inner ring to group together and points from the outer ring to group together.

The Result: The complex rings are mapped into linearly separable blobs in the spectral space. k-means achieves perfect separation.



Advantages & Disadvantages

Weighing the Costs and Benefits

Advantages

- Arbitrary Shapes: Exceptionally powerful at finding arbitrarily shaped and non-convex clusters.
- Global Optimization: Grounded in graph theory, offering a global optimization view (minimizing the graph cut) rather than just local density.

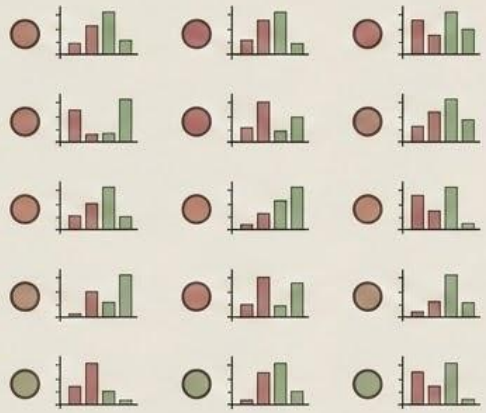
Disadvantages

- Computationally Expensive: Calculating eigenvectors for massive matrices is highly intensive. Time complexity is $O(n^3)$, making it difficult for very large datasets without approximation.
- High Sensitivity: The final results are highly sensitive to how the similarity graph is constructed and the parameters chosen (like σ in the Gaussian kernel).
- Choosing k: Determining the optimal number of clusters (k) remains a fundamental challenge.

Part 3: Clustering Graph and Network Data

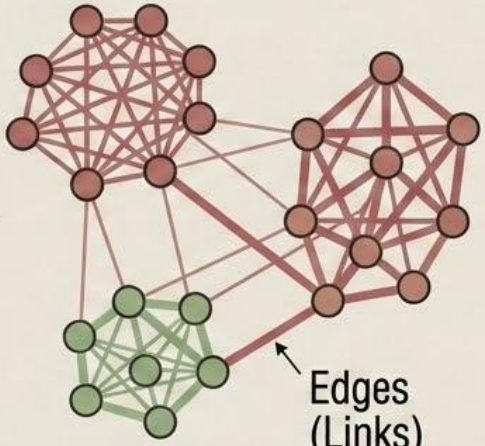
Shifting Focus from Entities to Relationships

Traditional Data View



Entities & Fixed Features

Graph Data View



Nodes (Vertices)

The Goal (Community Detection): Group nodes with dense internal connections, sparse external connections.

Defining Graph Data: Scenarios where relationships between entities (edges) are as important as, or more important than, the entities themselves (nodes).

Example 9.14. Bipartite graph. The customer purchase behavior at a company can be represented in a *bipartite graph*. In a bipartite graph, vertices can be divided into two disjoint sets so that each edge connects a vertex in one set to a vertex in the other set. For the customer purchase data, one set of vertices represents customers, with one customer per vertex. The other set represents products, with one product per vertex. An edge connects a customer to a product, representing the purchase of the product by the customer. Fig. 9.15 shows an illustration.

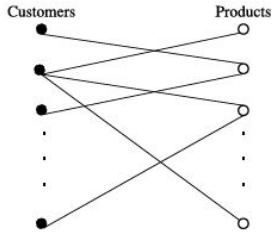


FIGURE 9.15

Bipartite graph representing customer-purchase data.

"What kind of knowledge can we obtain by a cluster analysis of the customer-product bipartite graph?" By clustering the customers such that those customers buying similar sets of products are placed into one group, a customer relationship manager can make product recommendations. For example, suppose Ada belongs to a customer cluster in which most of the customers purchased a popular game in the last 6 months, but Ada has yet to purchase one. As the manager, you can recommend the game to her.

Alternatively, we can cluster products such that those products purchased by similar sets of customers are grouped together. This clustering information can also be used for product recommendations. For example, if a robot vacuum and an electric smart cooker belong to the same product cluster, then when a customer purchases a robot vacuum, we can recommend the smart cooker. □

Example 9.15. Web search engines. In web search engines, search logs are archived to record user queries and the corresponding *click-through information*. (The click-through information tells us on which pages, given as a result of a search, the user clicked.) The query and click-through information can be represented using a bipartite graph, where the two sets of vertices correspond to queries and web pages, respectively. An edge links a query to a web page if a user clicks the web page when asking the query. Valuable information can be obtained by the cluster analysis on the query-web page bipartite graph. For instance, we may identify queries posed in different languages, but mean the same thing, if the click-through information for each query is similar.

As another example, all the web pages on the Web form a directed graph, also known as the *web graph*, where each web page is a vertex, and each hyperlink is an edge pointing from a source page to a destination page. Cluster analysis on the web graph can disclose communities, find hubs and authoritative web pages, and detect web spams. □

Example 9.16. Social network. A *social network* is a social structure and can be represented as a graph, where the vertices are individuals or organizations, and the links are interdependencies between the vertices, representing, for example, friendship, common interests, or collaborative activities. For a company, the customers may form a social network, where each customer is a vertex, and an edge links two customers if they know each other.

As a customer relationship manager, you are interested in finding useful information and knowledge that can be derived from the customer social network through clustering analysis. You obtain clusters from the network, where customers in a cluster know each other or have friends in common. Customers within a cluster may influence one another regarding purchase decision making. Moreover, communication channels can be designed to inform the "heads" of clusters (i.e., the "best" connected people in the clusters), so that promotional information can be spread out quickly. Thus you may use customer clustering to promote sales for the company.

As another example, the authors of scientific publications form a social network, where the authors are vertices and two authors are connected by an edge if they coauthored a publication. The network is, in general, a weighted graph because an edge between two authors can carry a weight representing the strength of the collaboration, such as how many publications the two authors (as the end vertices) coauthored. Clustering the coauthor network provides insight as to communities of authors and patterns

of collaboration. Indeed, in the context of (social) network analysis, clustering as finding well connected subgraphs is also known as *community detection*. □

Diverse Applications of Graph Clustering

Graph Clustering Applications: Diverse Domains, Unique Goals

Domain / Graph Type		Clustering (Community) Goal
1) Social Networks	User-follows-user / Friendship	Community detection, interest grouping.
2) Citation Networks	Paper-cites-paper	Identification of distinct research fields.
3) Web Graphs	Page-links-to-page	Automatic topic clustering of web pages.
Biological Networks	Protein-protein interaction	Functional module discovery (e.g., cell processes).
Transportation	City-road connections	Traffic flow and logical region analysis.

Key Challenges in Graph Clustering

Complexity at Scale

Scalability

Modern graphs are massive. They possess millions of nodes and billions of edges. Standard algorithms ($O(n^2)$ or $O(n^3)$) are computationally prohibitive; we need linear or near-linear time complexity.

Overlapping Communities

A fundamental reality. A node often belongs to multiple groups simultaneously (e.g., a social network user in family, work, and hobby groups). Traditional disjoint partitioning algorithms fail here.

Interpretability

Algorithms must do more than partition math; the resulting communities must be meaningful and actionable within the specific domain.

Dynamic Graphs

Real networks are not static; they evolve constantly over time. We need algorithms that can update clusters incrementally without re-running from scratch.

Node Similarity Metrics (Part 1)

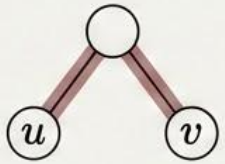
Measuring Node Similarity

Common Neighbors and Jaccard Index

The Goal: To cluster graphs effectively, we first need rigorous mathematical ways to measure exactly how "similar" two nodes (u and v) are based on their network connections.

Common Neighbors

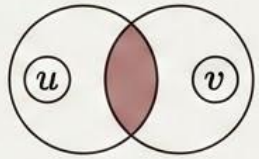
$$|\Gamma(u) \cap \Gamma(v)|$$



The count of shared neighbors between u and v.

Jaccard Index

$$\frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|}$$



The ratio of common neighbors to the total number of distinct neighbors across both nodes. (Normalized 0 to 1).

Note: $\Gamma(x)$ represents the set of neighbors for a given node x.

Example 9.17. Measurements based on geodesic distance. Consider graph G in Fig. 9.16. The eccentricity of a is 2. It is easy to verify that $eccen(a) = 2$, $eccen(b) = 2$, and $eccen(c) = eccen(d) = eccen(e) = 3$. Thus the radius of G is 2, and the diameter is 3. Note that it is not necessary that diameter $= 2\times$ radius. Vertices c, d, and e are peripheral vertices. □

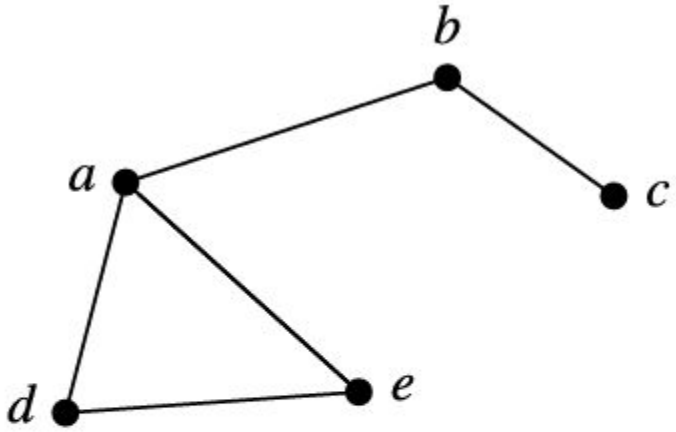


FIGURE 9.16

A graph, G, where vertices c, d, and e are peripheral.

Example 9.18. Similarity between people in a social network. Let us consider measuring the similarity between two vertices in the customer social network of a company, as discussed in Example 9.16. Here, similarity can be explained as the closeness between two participants in the network, that is, how close two people are in terms of the relationship represented by the social network.

"How well can the geodesic distance measure similarity and closeness in such a network?" Suppose Ada and Bob are two customers in the network, and the network is undirected. The geodesic distance (i.e., the length of the shortest path between Ada and Bob) is the shortest path that a message can be passed from Ada to Bob and vice versa. However, this information is not useful for the customer relationship management of the company, because the company typically does not want to send a specific message from one customer to another. Therefore geodesic distance does not suit the application.

"What does similarity mean in a social network?" We consider two ways to define similarity:

- Two customers are considered similar to each other if they have similar neighbors in the social network. This heuristic is intuitive because, in practice, two people receiving recommendations from a good number of common friends often make similar decisions. This kind of similarity is based on the local structure (i.e., the *neighborhoods*) of the vertices, and thus is called *structural context–based similarity*.
- Suppose the company sends promotional information to both Ada and Bob in the social network. Ada and Bob may randomly forward such information to their friends (or *neighbors*) in the network. The closeness between Ada and Bob can then be measured by the likelihood that other customers simultaneously receive the promotional information that was originally sent to Ada and Bob. This kind of similarity is based on the random walk reachability over the network, and thus is referred to as *similarity based on random walk*. □

Advanced Node Similarity Metrics (Part 2)

Weighting, Paths, and Probabilities

For complex networks, we need metrics that look beyond immediate, unweighted overlaps.

Adamic-Adar

$$\sum_{z \in \Gamma(u) \cap \Gamma(v)} \frac{1}{\log|\Gamma(z)|}$$

Weights common neighbors inversely by their degree, giving more importance to less connected shared neighbors.

Katz Index

$$\sum_{l=1}^{\infty} \beta^l (A^l)_{uv}$$

Looks beyond immediate friends. Counts the total number of paths of length l between nodes, penalizing longer paths using a decay factor (β).

Random Walk

Stationary Distribution

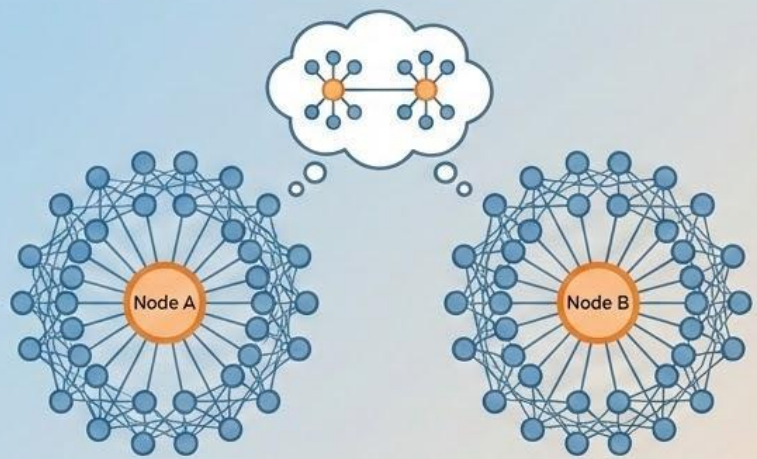
Measures similarity based on the probability of a random walker starting at node u and naturally landing on node v.

Note: $\Gamma(x)$ represents the set of neighbors for a given node x.

Network Equivalence

Similarity Beyond Direct Connections

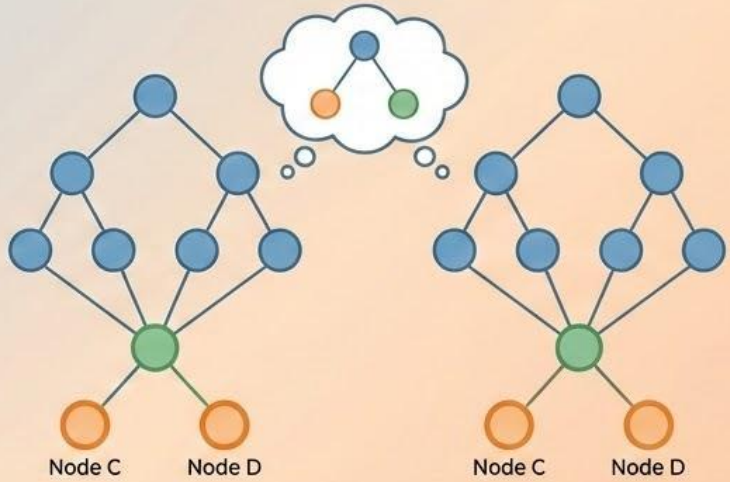
Structural Equivalence



Concept: Nodes are similar if they have highly similar connectivity patterns to the exact same overall groups.

Example: Two distinct “hub” nodes located in entirely different parts of a telecommunications network. They play the exact same structural role, even if they never interact.

Regular Equivalence



Concept: Nodes are similar if they connect to similar types of nodes, regardless of identity.

Example: Two “leaf” nodes (nodes with only one connection) situated in two completely different data trees. They are equivalent in their functional hierarchy.

Example 9.19. Cuts and clusters. Consider graph G in Fig. 9.17. The graph has two clusters: $\{a, b, c, d, e, f\}$ and $\{g, h, i, j, k\}$, and one outlier vertex, l.

Consider cut $C_1 = (\{a, b, c, d, e, f, g, h, i, j, k\}, \{l\})$. Only one edge, namely, (e, l), crosses the two partitions created by C_1. Therefore the cut set of C_1 is $\{(e, l)\}$, and the size of C_1 is 1. (Note that the size of any cut in a connected graph cannot be smaller than 1.) As a minimum cut, C_1 does not lead to a good clustering because it only separates the outlier vertex, l, from the rest of the graph.

Cut $C_2 = (\{a, b, c, d, e, f, l\}, \{g, h, i, j, k\})$ leads to a much better clustering than C_1. The edges in the cut set of C_2 are those connecting the two "natural clusters" in the graph. Specifically, for edges (d, h) and (e, k) that are in the cut set, most of the edges connecting d, h, e, and k belong to one cluster. □

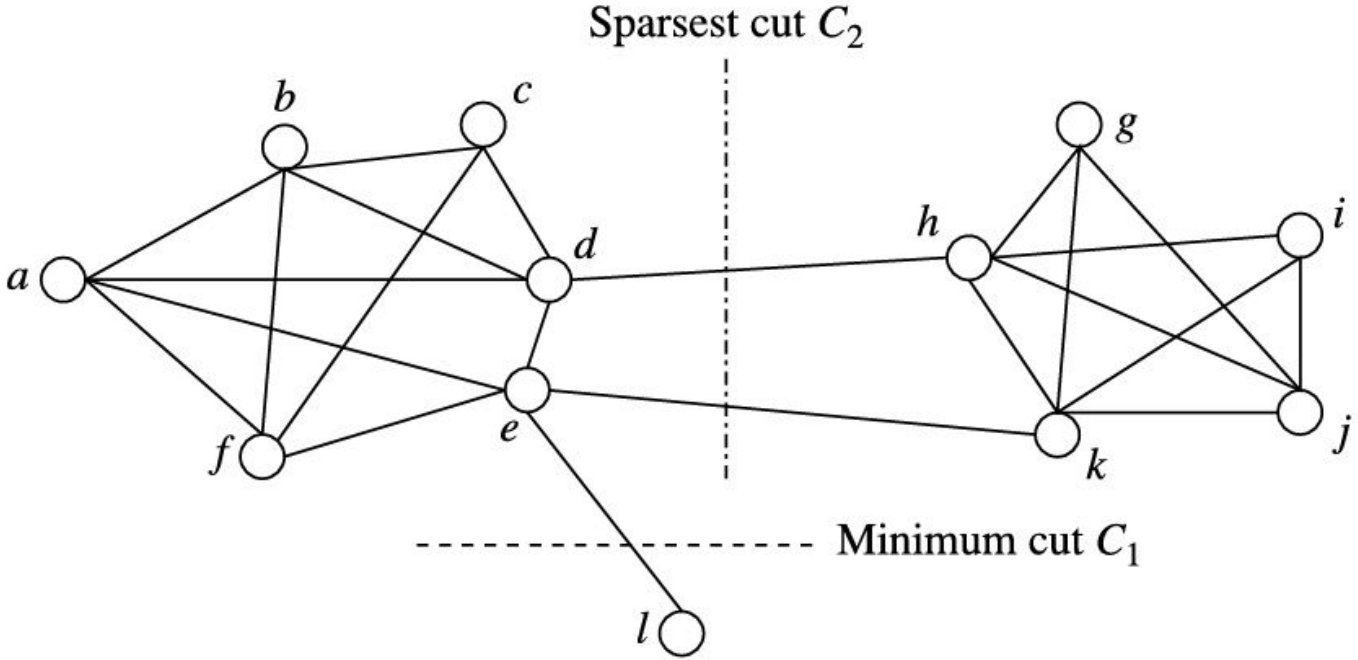


FIGURE 9.17

A graph G and two cuts.

Method 1: Modularity Maximization

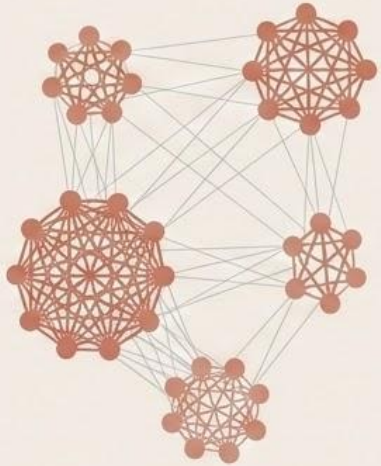
Optimizing Community Structure

The Concept

Instead of just grouping nodes, we measure the quality of a network partition using a metric called Modularity (Q).

The Modularity Formula

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i \cdot k_j}{2m} \right] \cdot \delta(c_i, c_j)$$



What it Means

Higher Q values indicate a stronger, better community structure (dense connections inside, sparse connections outside).

The Louvain Algorithm

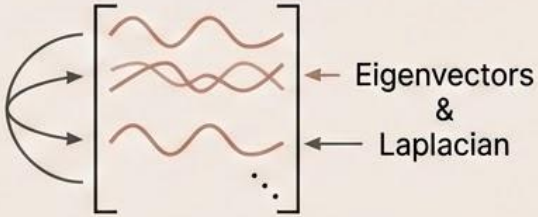
A highly popular, greedy optimization method used to rapidly maximize modularity in large networks.

Methods 2 & 3: Spectral and Label Propagation

Laplacians and Majority Rules

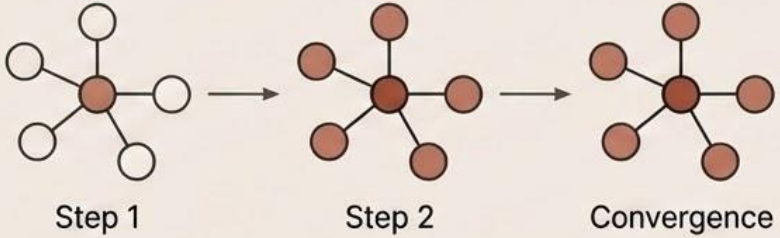
2. Spectral Graph Clustering

- **Approach:** Applies standard spectral clustering (eigenvectors) directly to the graph Laplacian.
- **Strengths:** Exceptionally good at identifying deeply cohesive groups. Can be scaled moderately using sparse matrix methods.



3. Label Propagation

- **Approach:** A viral, localized method. Every node looks at its neighbors and simply adopts the label held by the majority.
- **Process:** This repeats iteratively until the network stabilizes (convergence).
- **Strengths:** Incredibly fast with near-linear complexity.

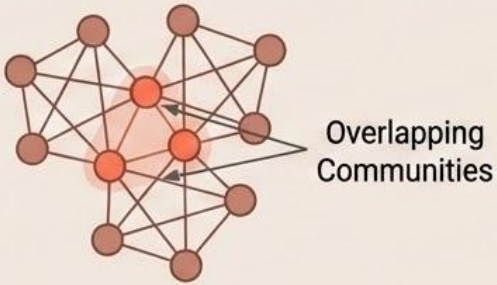


Methods 4 & 5: Cliques and Edge Removal

Capturing Overlaps and Bottlenecks

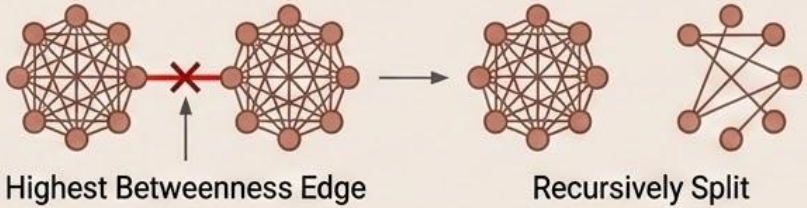
4. Clique Percolation Method (CPM)

- **Approach:** Hunts for all k-cliques (subgraphs where every single node is connected to every other node). Communities are formed where these dense cliques overlap.
- **Strengths:** One of the best methods for explicitly identifying overlapping communities.



5. Betweenness-Based Methods (Girvan-Newman)

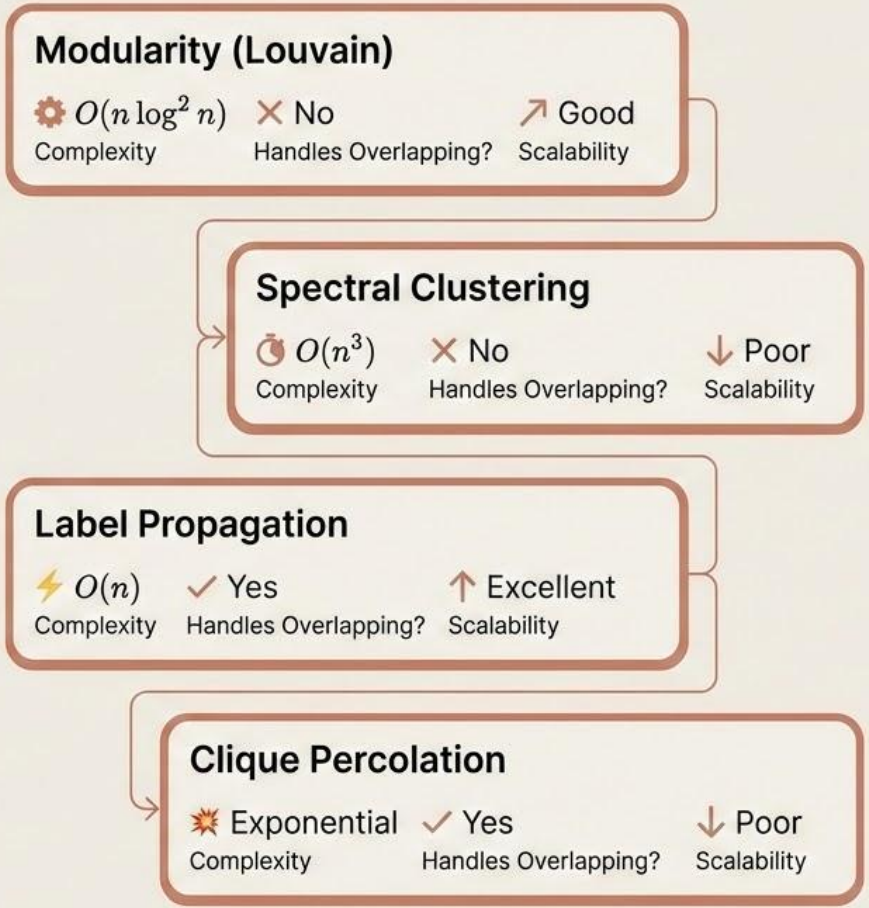
- **Approach:** A top-down, divisive approach. It computes "edge betweenness centrality" (finding the "bridge" edges connecting different communities).
- **Process:** It repeatedly removes the edges with the highest betweenness, recursively splitting the network into smaller components.



Comparing Graph Clustering Methods

Complexity, Overlap, and Scalability

Choosing the right algorithm depends entirely on your dataset size and whether you need to capture overlapping groups.



Part 4: Semi-Supervised Clustering

Incorporating Partially Labeled Data

The Fundamental Problem:

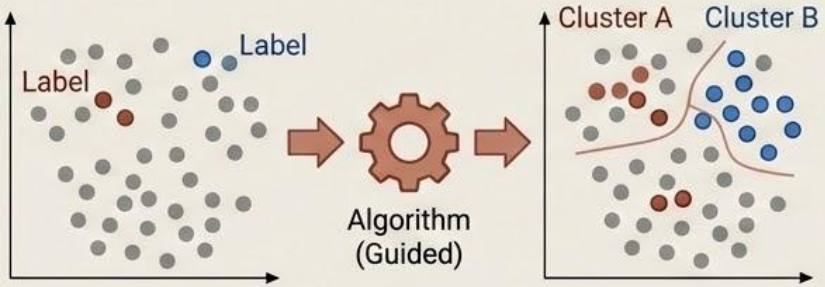
Traditional fully unsupervised clustering assumes zero prior knowledge, completely ignoring any valuable, authoritative labeled information that might actually be readily available.

The Opportunity:

Why guess when you know? Often, a small fraction of your dataset might already possess established labels (e.g., from an expert review or a previous task).

The Goal:

To fundamentally improve clustering quality by actively incorporating this limited, high-**value** labeled data as guidance to steer the algorithm toward a more accurate and interpretable result.



Example 9.20. Clustering with partially labeled data. Imagine that you want to build a malware detector. You collect a large number of images of host systems. You and some of your colleagues label a small number of such images into two categories: malware and benign. However, there is no hope that all or even a fair portion of such images are labeled properly and timely. What should you make good use of the data?

An immediate idea is to use the labeled data to build a classifier. However, the proportion of labeled data is very small. A classifier built on such a small amount of data may not be effective and cannot use a large amount of unlabeled data. In other words, the vast majority of the data available cannot be used by a classification method. □

Core Approaches to Semi-Supervision

Constraints, Metrics, and Initialization

We can inject this limited labeled data into the clustering process using three primary methodologies:

1. Constraint-Based

How Labeled Data Is Used:
Labeled points are used to define strict rules that the final partition must follow.

Key Mechanism/Examples:

- **Must-link**: Points known to be similar must be in the same cluster.
- **Cannot-link**: Points known to be different must be in different clusters.

2. Distance Metric Learning

How Labeled Data Is Used:
Labeled points are used to train a customized distance function that better reflects similarity.

Key Mechanism/Examples:

- **Relevant Component Analysis (RCA)**: Learns a new Mahalanobis distance metric based on labeled subsets before clustering begins.

3. Initialization-Based

How Labeled Data Is Used:
Labeled points are used to provide intelligent "hints" for the algorithm's starting state.

Key Mechanism/Examples:

- **Semi-Supervised k-Means**: Each labeled point directly initializes a cluster centroid, giving the algorithm an expert starting position.

Semi-Supervised k-Means Algorithm

A Specific Example of Initialization-Based Semi-Supervision

Input

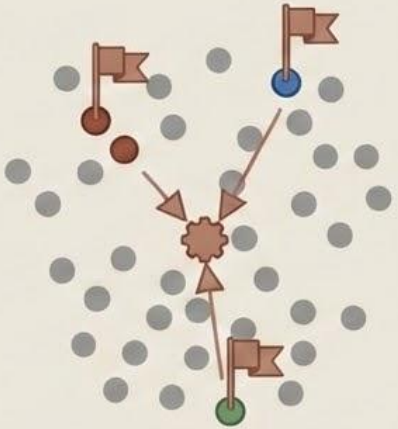
- Data points (X)
- Specific labeled points with known cluster assignments
- Total set of unlabeled points
- Target number of clusters (K)

Output

- Cohesive and authoritative cluster assignments

The Algorithm Steps

1. Initialize centroids authoritatively using only the labeled points.
2. For each unlabeled point, calculate distance and assign to its nearest centroid.
3. Update all centroids as the mathematical mean of ALL assigned points (labeled + unlabeled).
4. Repeat Steps 2 & 3 iteratively until the centroids converge.

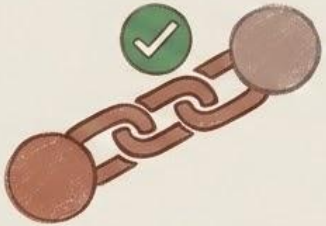


Guiding Algorithms with Pairwise Constraints

Must-Link and Cannot-Link

The Concept: Instead of labeling individual points (e.g., 'This is a cat'), we provide the algorithm with relational rules between specific pairs of points.

Types of Constraints:



Must-Link (ML)
Two points must be assigned to the **exact same** cluster.

Example:
Two different photos that we know for a fact feature the exact same person.

Cannot-Link (CL)
Two points must be assigned to **different clusters**.

Example:
Two photos that we know feature two **completely different** people, even if they look similar.

Example 9.21. Clustering with pairwise constraints. Suppose as a customer relationship manager of a wholesale company, you want to use a clustering method to divide the representatives of your customer companies into several groups so that a marketing campaign event is run for a group. You may have multiple representatives from the same customer company. They should be invited to the same event. In order to express such domain knowledge, you want to specify must-link constraints between every pair of representatives who should be invited together.

Moreover, you may have two customer companies that are head-to-head competitors. You may want to make sure that representatives from two head-to-head competitors are not invited to the same event. Accordingly, you want to specify cannot-link constraints between every pair of representatives who should not be invited to the same event. □

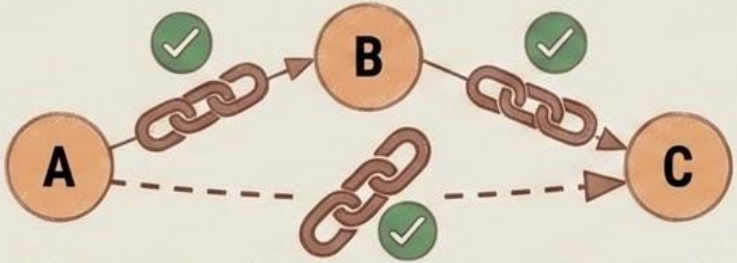
Constraint Propagation

Maximizing Limited Information

The Goal: To logically deduce new constraints from the limited set of rules provided by the user, expanding the algorithm's knowledge base automatically.

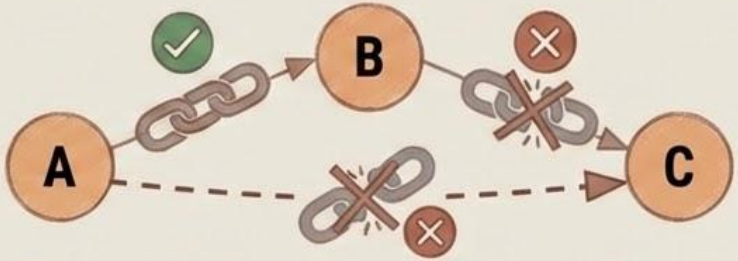
Transitive Closure

Rule 1: Must-Link + Must-Link = Must-Link



(If A must link to B, and B must link to C, then A must link to C)

Rule 2: Must-Link + Cannot-Link = Cannot-Link



(If A must link to B, but B cannot link to C, then A cannot link to C)

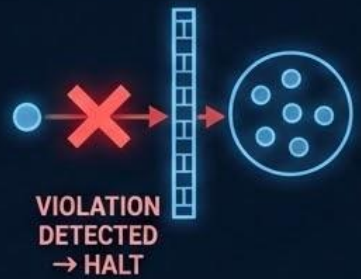
Adapting k-Means for Constraints

Hard vs. Soft Rule Enforcement

How do standard algorithms handle these new rules?

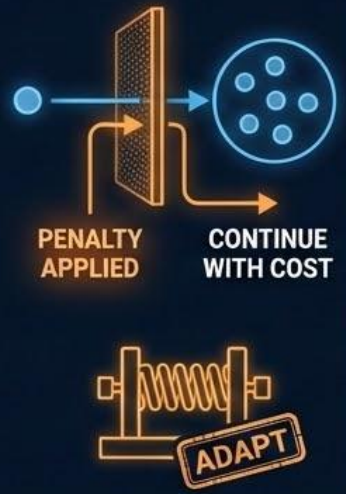
COP-KMeans (Hard Constraints)

- A modified version of k-means that strictly respects all constraints.
- During the assignment phase, it will never violate a rule.
- If no feasible assignment exists that satisfies all rules, the algorithm will intentionally fail and halt.



PCKMeans (Soft Constraints)

- Stands for Pairwise Constrained K-Means.
- Treats rules as guidelines rather than absolute laws.
- It mathematically penalizes the algorithm for violating constraints.
- Highly effective when constraints might be noisy, contradictory, or inconsistent.



Spectral Learning with Constraints

Rewiring the Similarity Matrix

The Strategy:

We can seamlessly integrate pairwise constraints into advanced graph-based methods like Spectral Clustering.

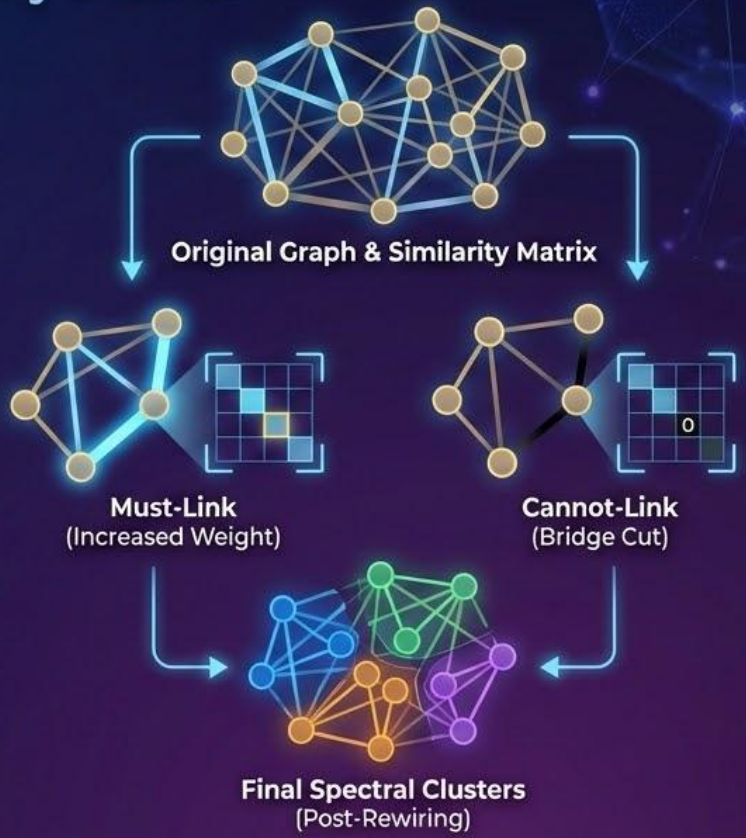
Modifying the Graph:

Must-Link: Artificially increase the weight/similarity between the two nodes in the similarity matrix (drawing them closer).

Cannot-Link: Artificially decrease the weight/similarity to zero between the two nodes (cutting the bridge between them).

Execution:

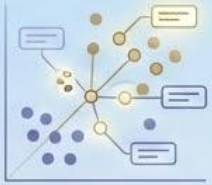
Once the similarity matrix is fundamentally "rewired" by these constraints, apply standard spectral clustering to extract the final groups.



Beyond Just Constraints

Types of Background Knowledge: Part 1 - Instance and Feature-Level Guidance

1. Instance-Level Knowledge

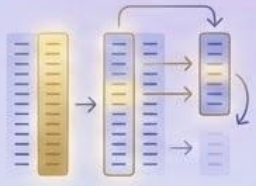


Partial Labels: Knowing the exact class membership for a small subset of points.

Pairwise Constraints: The Must-Link and Cannot-Link rules we just discussed.

Guidance applied to specific, individual data points.

2. Feature-Level Knowledge



Feature Relevance: Explicitly telling the algorithm that certain features are vastly more important than others.

Feature Relationships: Providing known correlations or hierarchical dependencies between variables.

$f(x)$ **Feature Transformations:** Applying known mathematical mappings to the data before clustering begins.

Types of Background Knowledge: Part 2

Cluster Structure and Domain Integration

3. Cluster-Level Knowledge

Guidance on the overall, macro-level shape of the final results.

- **Expected Size Ranges:** Setting rules (e.g., 'No cluster can have fewer than 50 points').

- **Cluster Structure:** Defining expected levels of compactness or specific geometric shapes.

- **Count Limits:** Setting strict minimum or maximum limits on the total number of clusters allowed.

4. Domain Knowledge Integration

Leveraging external, structured expertise.

- **Ontologies:** Plugging in structured, industry-standard domain knowledge (like medical ontologies).

- **Taxonomies:** Utilizing established hierarchical relationships (e.g., biological classifications).

- **Seeding:** Using these existing taxonomies to intelligently jump-start the initial clustering process.

The Value of Semi-Supervised Clustering

Why We Incorporate Background Knowledge

Improved Accuracy

Consistently outperforms purely unsupervised methods by avoiding mathematical "blind spots."

Enhanced Interpretability

The resulting clusters naturally align much closer to human logic, business realities, and domain expectations.

High Efficiency

Achieves near-supervised levels of performance while drastically reducing the need (and massive cost) for manually labeling entire datasets.

Robustness to Noise

Background knowledge acts as an anchor, preventing the algorithm from being derailed by noisy, messy, or outlier-heavy data.

Summary: Method Comparison

Inputs, Strengths, and Limitations

The Landscape: No single algorithm solves every problem. Choosing the right approach requires balancing the data structure against computational limits.

Method	Required Input	Key Strength	Key Limitation
NMF	Feature matrix	Highly interpretable, parts-based	Linear relationships only
Spectral	Similarity matrix	Solves non-convex/complex clusters	$O(n^3)$ computational complexity
Modularity	Graph / Network	Identifies strong communities	Resolution limit (misses small groups)
Label Propagation	Graph / Network	Extremely fast, handles overlapping	Inherent randomness in results
Semi-Supervised	Features + constraints	Maximizes available expert labels	Requires accurate constraints/labels

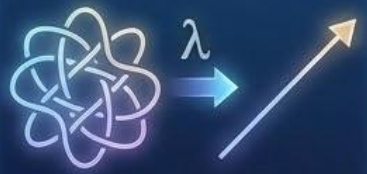
Strategic Selection

When to Use Which Method

Real-World Scenario	Recommended Method
 Parts discovery (e.g., Image features or text topics)	NMF
 Complex, non-convex shapes (Moderate dataset size)	Spectral Clustering
 Massive-scale analysis (e.g., Global social networks)	Label Propagation
 Known structural communities (e.g., Web link grouping)	Modularity Maximization
 Expert knowledge available (Limited labels or constraints)	Semi-Supervised Clustering

Key Insights & Conclusion

The Evolution of Clustering



Geometry Translation

Spectral clustering elegantly transforms impossible, non-linear geometric problems into simple, linear ones via eigen-decomposition.

Relationships Over Attributes

Graph clustering requires a fundamental shift in thinking—understanding network structure and similarity metrics is often more important than the node attributes themselves.

Bridging the Gap

Semi-supervised clustering bridges the massive gap between fully supervised classification and purely unsupervised exploration.

The Ultimate Advantage

Mathematical algorithms are powerful, but domain knowledge significantly and consistently improves clustering quality when integrated properly.